Passing Arrays and Slices

We already know that Go uses **Call by Value** (making a copy of the data) when passing arguments into functions. When dealing with lists of data, this default behavior creates a massive performance trap if you aren't careful.

1. The Problem with Passing Arrays

If you have an array of **100,000** integers and you pass it into a function, Go will literally copy all **100,000** integers into a brand-new array in memory just for that function to use.

- **The Result:** It wastes CPU time, duplicates memory usage, and slows down your application. Furthermore, any changes the function makes to the array are only applied to the copy, not the original.

2. The "Messy" Solution: Array Pointers

To avoid duplicating the massive array, you could manually pass a pointer to the array (Call by Reference).

- **How it looks:** You define the function parameter as a pointer **func foo(x *[3]int)** and pass the array using the ampersand **foo(&a).**
- **The Result:** It solves the memory problem, and the function can now modify the original array. However, the syntax gets very messy because you have to constantly use * and & to reference and dereference the data. **This is not the "Go Way".**

```
func foo(x *[3]int) { // Messy and Confusing Syntax
    (*x)[0] = (*x)[0] + 1
}

func main() {
    a := [3]int{1, 2, 3}
    foo(&a)
    fmt.Print(a)
}
```

3. The Go Standard: Always Pass Slices!

In Go, the golden rule is to almost always use **Slices** instead of fixed Arrays, especially when passing data between functions.

Here is the secret to why Slices are so powerful:

- A Slice is not the actual data. It is just a tiny, lightweight **"struct"** containing exactly three things: a **Pointer** to the underlying array, the **Length**, and the **Capacity**.
- When you pass a Slice into a function, Go's Call by Value only copies that tiny 3-part struct (which is extremely fast and takes almost zero memory).

- Because the copied struct still contains the exact same memory **Pointer** looking at the exact same underlying array, the function can directly modify the original data without any messy *** or & syntax!**

```
func foo(sli []int) int { // Passing Slice
    sli[0] = sli[0] + 1
}

func main() {
    a := []int{1, 2, 3}
    foo(a)
    fmt.Print(a)
}
```

Note: The difference in syntax between **arrays** and **slices** is brilliantly simple:

- **Array: a := [3]int{1, 2, 3}** (You lock in the size).
- **Slice: a := []int{1, 2, 3}** (You leave the brackets empty).

Expert Takeaway:

When scaling a backend architecture, **minimizing "Garbage Collection"** pauses is key to keeping latency low. If your server is constantly duplicating massive lists of database rows just to pass them through validation functions, the server will choke on memory.

By natively using **Slices** to pass data between your controllers, services, and repository layers, you guarantee that your application is only passing around tiny 24-byte pointers, keeping your API response times lightning fast!